# Lab 08

## Recap: Lab (Github) Workflow 🔄 - How to Work on Labs

Follow these steps for every lab carefully to access, complete, and submit your assignment.

1. **Accept the Assignment**
   - Open the Lab Assignment Link the professor provided.
   - Click **"Accept the assignment"**. This will create your personal assignment repository on GitHub under the `OOP-Fall-2025` organization.
   - You'll be taken to your repository page. Verify that the URL looks like `github.com/OOP-Fall-2025/lab-number-yourusername`.
2. **Clone the Repository to Your Computer**
   - On your repository page, click the blue `<> Code` button.
   - In the dropdown menu, choose **"Open with GitHub Desktop"**.
   - GitHub Desktop will launch. Choose a preferred local folder on your computer to save the project and click **"Clone"**.
   - If asked "How are you planning on using this fork?", select **"For my own purpose"** and continue.
3. **Open in VS Code and Start Coding**
   - In GitHub Desktop, ensure the "Current repository" is the one for this lab.
   - Click the **"Open in Visual Studio Code"** button.
   - VS Code will open the project folder. You can now begin writing your solutions in the `Lab08.java` file.
4. **Save and Submit Your Work**
   - **Commit (Save) Changes**: As you work, save your file in VS Code (`Ctrl+S` or `Cmd+S`). To record your progress, go to the **Source Control** tab (the fork icon) on the left sidebar in VS Code. Type a descriptive message in the message box (e.g., "Finished Task 1 and 2") and click **"Commit"**. You must enter a message.
   - **Push (Submit) to GitHub**: When you are finished with the lab or want to back up your work, go back to GitHub Desktop. Click the **"Push origin"** button at the top of the window. This sends your committed changes from your computer to your GitHub repository online.
5. **Verify Your Submission**
   - After you push, you can click **"View on GitHub"** in GitHub Desktop to open your repository in the browser.
   - On the GitHub website, make sure you are viewing the `main` branch and confirm that all of your latest code is visible.

---

## Lab 08 Tasks

### Shapeshifting

*Same contract, different performances*

## Task 1: Animal Kingdom

**Topics Covered:** Interfaces, Overriding, Overloading

**What This Task Practices:** You'll create animals that can speak using an interface, override their behaviors, and use overloaded constructors to create objects in different ways.

You will create 1 interface and 3 classes:

### Speakable (Interface)

- Create an interface called `Speakable`
- Add one method signature: `void makeSound()`

### Animal

- This is the base class
- **Implements** the `Speakable` interface
- **Property**: `name` (String) - The name of the animal
- **Property**: `age` (int) - The age of the animal
- **Overloaded Constructor #1**: Takes only `name` as parameter, sets name and sets age to 0
- **Overloaded Constructor #2**: Takes `name` and `age` as parameters, sets both properties
- **Method**: `toString()` that **returns** the string `"[name] is an animal!"`
  - Use `@Override` on the `toString()` method
- **Method**: `makeSound()` that prints `"[name] makes a sound!"`
  - Use `@Override` on the `makeSound()` method

### Duck

- Inherits from Animal and implements Speakable (inherited from Animal)
- **Overloaded Constructor #1**: Takes only `name`, calls `super(name)`
- **Overloaded Constructor #2**: Takes `name` and `age`, calls `super(name, age)`
- **Override** `toString()` to **return** the string `"[name] is a duck!"`
- **Override** `makeSound()` to print `"[name] quacks!"`

### Sloth

- Inherits from Animal and implements Speakable (inherited from Animal)
- **Overloaded Constructor #1**: Takes only `name`, calls `super(name)`
- **Overloaded Constructor #2**: Takes `name` and `age`, calls `super(name, age)`
- **Override** `toString()` to **return** the string `"[name] is a sloth!"`
- **Override** `makeSound()` to print `"[name] moves slowly and says nothing."`

### Main

- Ask for the Duck's name

- Create a Duck object using the constructor with just name
- Ask for the Sloth's name and age
- Create a Sloth object using the constructor with name and age
- Print the Duck object (tests toString override)
- Call makeSound() on the Duck (tests interface implementation)
- Print the Sloth object (tests toString override)
- Call makeSound() on the Sloth (tests interface implementation)

### Example Output

```
Duck Name: Quackers

Sloth Name: Velcro

Sloth Age: 5

Quackers is a duck!

Quackers quacks!

Velcro is a sloth!

Velcro moves slowly and says nothing.
```

---

**(There's more...)**

*Everyone plays their part*

## Task 2: Rock and Roll All Night

**Topics Covered:** Interfaces, Polymorphism, Overriding, Overloading, Inheritance

**What This Task Practices:** You'll build a band where different musicians inherit from a base class, implement a performer interface, and demonstrate polymorphism by storing different types of musicians in one array.

You will create 1 interface and 7 classes.

### Performer (Interface)

- Create an interface called `Performer`
- Add one method signature: `void perform()`

### Musician

- **Implements** the `Performer` interface
- **Public property**: `name` (String) - The name of the musician
- **Overloaded Method #1**: `play()` - Returns `"[name] plays music."`

- **Overloaded Method #2**: `play(String song)` - Returns `"[name] plays [song]."`
- **Method**: `perform()` - Prints `"[name] is performing!"`
    - Use `@Override` on the `perform()` method

### Bassist

- Inherits from Musician
- **Override** `play()` to return: `"[name] plays bass."`

### Drummer

- Inherits from Musician
- **Override** `play()` to return: `"[name] plays drums."`

### Guitarist

- Inherits from Musician
- **Override** `play()` to return: `"[name] plays guitar."`

### RhythmGuitarist

- Inherits from Guitarist
- **Override** `play()` to return: `"[name] plays rhythm."`

### LeadGuitarist

- Inherits from Guitarist
- **Override** `play()` to return: `"[name] plays a mean solo!"`

### Main

- Create an array of Musician (size 4)
- For each Musician, ask the name and kind of musician (Bassist, Drummer, RhythmGuitarist, LeadGuitarist)
- Create the appropriate type of Musician and set the name

After you have the band...

- Loop through the band
- Print: `band[i].play()` (demonstrates polymorphism and overriding)
- Call: `band[i].perform()` (demonstrates interface implementation)

### Example Output

```
Name: Ringo

Kind: Drummer

Name: Paul
```

```
Kind: Bassist

Name: George

Kind: RhythmGuitarist

Name: John

Kind: LeadGuitarist

Ringo plays drums.

Ringo is performing!

Paul plays bass.

Paul is performing!

George plays rhythm.

George is performing!

John plays a mean solo!

John is performing!
```

---

## Key Concepts Practiced:

- **Interfaces**: Creating and implementing contracts (Speakable, Performer)
- **Overriding**: Using @Override to change parent class behavior
- **Overloading**: Multiple constructors/methods with different parameters
- **Polymorphism**: Using parent class references to hold child objects

---

## Need Help?

Ask me for help! I'm literally in the same room as you!

## Finished?

When you are done with the labs (finished and committed on GitHub properly), call me over and show me. Tell me your name and I'll mark you as done!